

Tutorial 6 - More Maximum Likelihood

SFR605

This week's exercise will continue our exploration of analytical and numerical approaches to fitting models by maximum likelihood. Most of the techniques will remain the same but we will be looking at a model of greater complexity.

#Load needed libraries

```
library(here)
```

```
## here() starts at C:/Users/sande/OneDrive/Desktop/Code/SFR605
```

Case Study: Effects of Elevated CO2 on Pine Fecundity

The response of ecosystems to elevated CO2 involves not just the change in growth rates of plants, but also the change in their survival and reproduction. This week's analysis will look at the response of loblolly pine (*Pinus taeda*) to being grown in both elevated and ambient CO2 under field conditions at a FACE (Free Air CO2 Enrichment) site in North Carolina (LaDeau and Clark 2001). Part of this data set consists of counts of the number of pine cones on trees in both treatments. Trees of different sizes are known to vary in their reproductive output, with larger trees being more fecund. Because we know that the trees under elevated CO2 are growing faster, and thus are on average larger, this analysis will need to account for tree size as a covariate to determine if trees in elevated CO2 produce more seed at a given size (i.e. fecundity | size) or if the increase in fecundity under elevated CO2 is just due to an increase in tree size. This experiment fortuitously began right as the trees were starting to mature, so there is the additional question of whether CO2 affects the size at which trees reach maturity.

Lets begin by loading up the data and taking a look at it

```
b <- read.csv(here("data/pinecones.tidy.txt"),header=TRUE)
```

```
str(b)
```

```
## 'data.frame':    1932 obs. of  6 variables:
## $ ring : int  1 1 1 1 1 1 1 1 1 1 ...
## $ tmt  : chr  "AMB" "AMB" "AMB" "AMB" ...
## $ id   : int  1 1 1 2 2 2 3 3 3 4 ...
## $ year : int  1998 1999 2000 1998 1999 2000 1998 1999 2000 1998 ...
## $ diam : num  19.4 19.4 19.4 14.1 14.1 14.1 19.4 19.4 19.4 21.6 ...
## $ cones: int  0 0 0 0 0 4 0 6 0 0 ...
```

```
summary(b)
```

```
##      ring      tmt      id      year
```

```
## Min. :1.000 Length:1932 Min. : 1.0 Min. :1998
## 1st Qu.:2.000 Class :character 1st Qu.:161.8 1st Qu.:1998
## Median :4.000 Mode :character Median :322.5 Median :1999
## Mean :3.781 Mean :322.5 Mean :1999
## 3rd Qu.:5.000 3rd Qu.:483.2 3rd Qu.:2000
## Max. :6.000 Max. :644.0 Max. :2000
## diam cones
## Min. : 2.60 Min. : 0.000
## 1st Qu.:12.57 1st Qu.: 0.000
## Median :16.85 Median : 0.000
## Mean :15.95 Mean : 1.428
## 3rd Qu.:19.60 3rd Qu.: 0.000
## Max. :28.60 Max. :91.000
```

```
table(b$ring,b$tmt) ## shows number of trees by ring and treatment
```

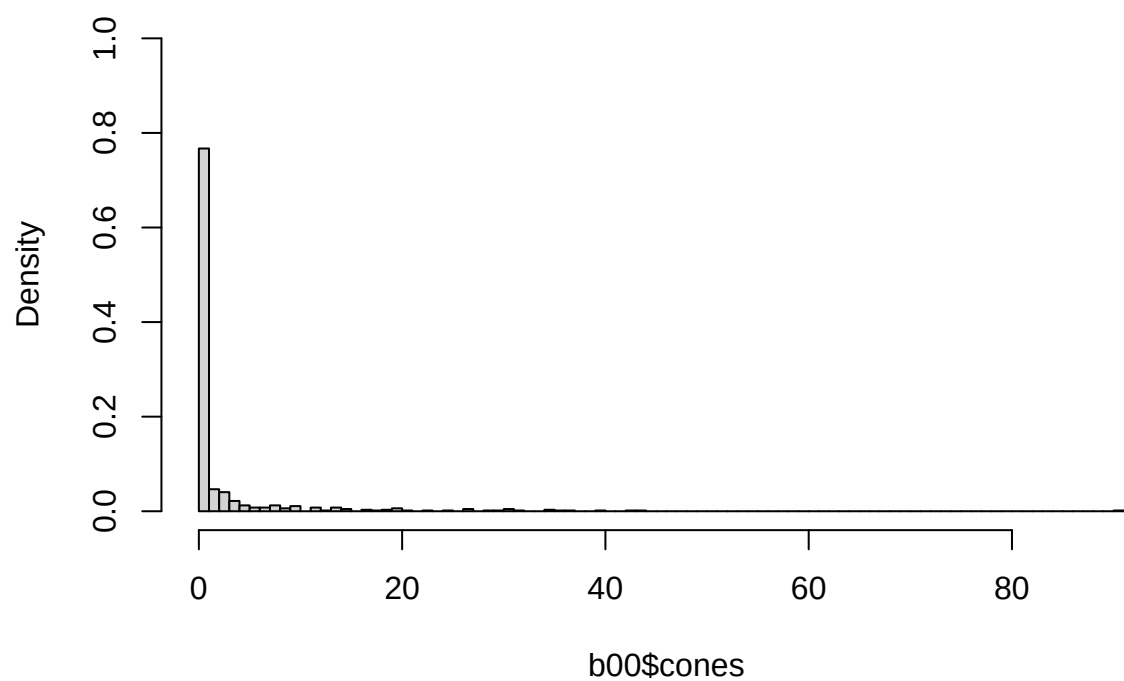
```
##
## AMB CO2
## 1 288 0
## 2 0 213
## 3 0 327
## 4 0 351
## 5 312 0
## 6 441 0
```

```
b00 = b |> dplyr::filter(year==2000)
```

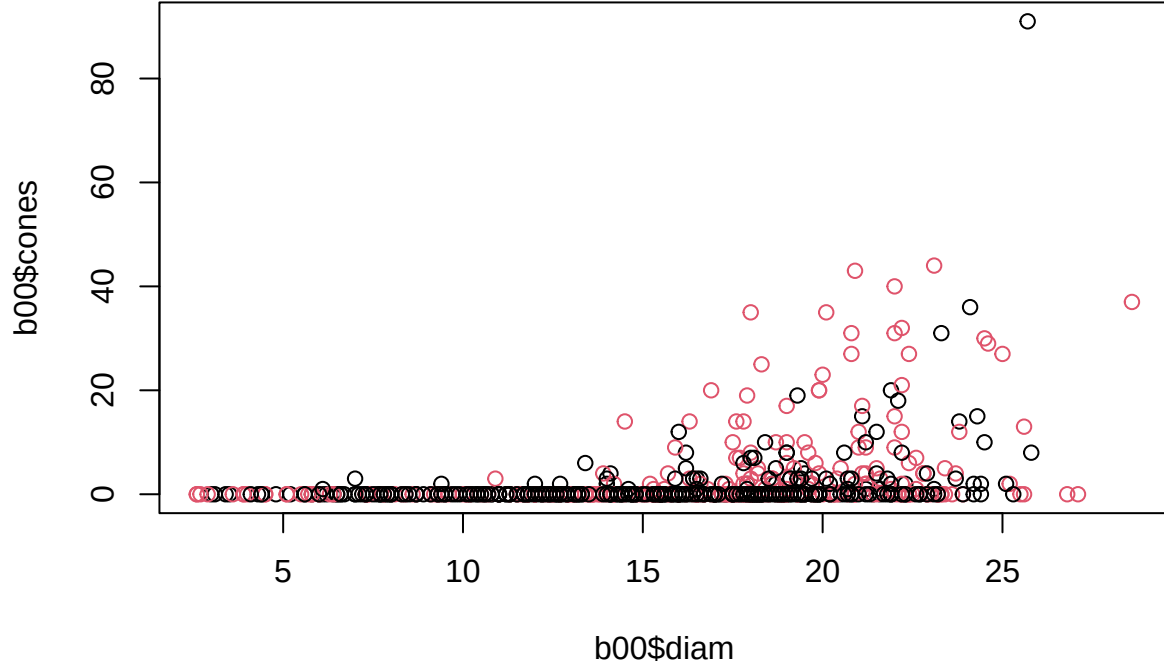
```
##histogram of cone counts in year 2000
```

```
hist(b00$cones,breaks=0:92,probability=TRUE,ylim=c(0,1))
```

Histogram of b00\$cones



```
##plot of counts by tree size (x-axis) and CO2 (AMB is black, CO2 is red)
plot(b00$diam,b00$cones,col = as.factor(b00$tmt))
```



This data set represents observations of individual trees over time. The first column in the data set (ring) is an indicator of which plot the tree was in – there are 6 circular plots at the FACE experiment. The second (tmt) indicated which treatments were applied to which rings, where AMB refers to ambient CO2 concentrations and CO2 refers to elevated CO2. The next column is an ID number (id) for the tree, which then followed by the year it was measured (year), and the diameter of the tree at 1.3 m height (diam = diameter breast height) in centimeters. Other measures of tree size (height, mass, etc.) are highly correlated with diameter. The final column (cones) is the cone counts. To keep the analysis more manageable we'll focus on just the data from 2000.

Fecundity Model

For the first phase of our analysis let's focus on the relationship between fecundity (cone count) and tree size (diameter). Since the number of cones that are produced on a mature individual is positive count data let's assume that cone production, y , is Poisson

$$L = \prod_{i=1}^n \text{Pois}(y_i | g(x_i))$$

where the expected number of cones per tree, g , is a function of tree diameter. Now that we have specified the data model for our analysis we next need to specify the process model, which is the function $g(x)$. For $g(x)$ there is some evidence that fecundity increases as the square of diameter, so we will model g as

$$g(x_i) = a_0 x_i^2$$

Before we get to fitting the model, lets define some variables that specify what data is going to be used in the fitting

```
dia   <- b00$diam      ## tree diameters
ncone <- b00$cones      ## number of cones in 2000
cones <- (ncone > 0)    ## whether cones are present - this indicates whether tree is mature
tmt   <- b00$tmt       ## treatment (elevated vs ambient)
```

Next, lets define our negative log likelihood function for this model

```
##' @param a0    unknown parameter (slope of fecundity/size relationship)
##' @param x     tree diameter at breast height (cm)
##' @param y     cone counts (integer)
likFec = function(a0,x,y){
  g <- a0*x^2          ## expected mean fecundity, g(x),as a function of tree size
  -sum(dpois(y,g,log=TRUE)) ## negative log likelihood
}
```

To try and make the distinction between the data model and the process model clear we've split this R function into two lines, one that calculates $g(x)$, the expected number of cones for a tree of a given size, and the second that calculated the negative log likelihood.

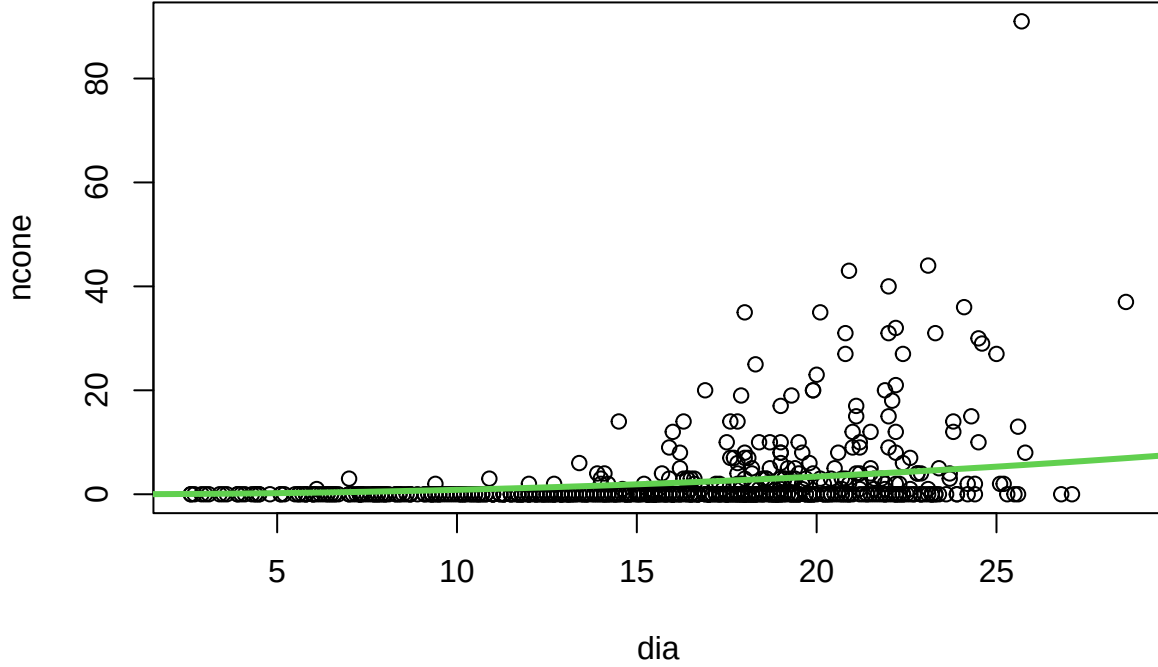
Now that we have our likelihood function, we'll want to fit it numerically and then take a look at the results, both in terms of the numbers returned and in a basic graph of the fit of the model. As a first pass, lets consider all data from both treatments together

```
out.fec <- optimize(likFec,c(0.001,1),x=dia,y=ncone)
out.fec
```

```
## $minimum
## [1] 0.008494147
##
## $objective
## [1] 2563.153
```

```
a0 <- out.fec$minimum ## maximum likelihood parameter estimate

plot(dia,ncone)          ## raw data scatter plot
dseq <- seq(1,30,length=300) ## seq of x values for drawing curve
lines(dseq,a0*dseq^2,col=3,lwd=3) ## add fecundity relationship
```



Maturation Model

From the exploratory data analysis and this initial model of tree fecundity it is clear that there are a large number of trees with a cone count of zero every year and that smaller trees are likely to be reproductively immature. Therefore our data model needs to account for both the status of the plant (reproductively mature vs. immature) and the variability associated with the count data of the number of cones. We have already discussed the later part in the fecundity model above so let's take a look at reproductive maturation. Since maturation is boolean in nature, but a function of tree size, let's treat it as a Bernoulli process:

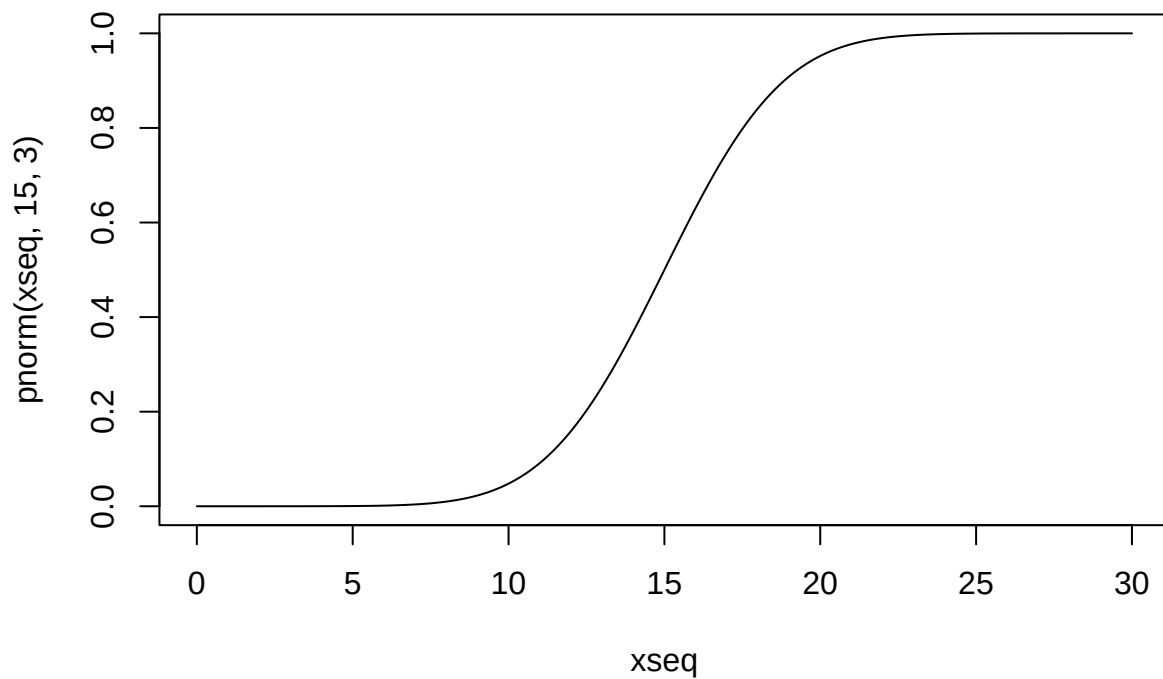
$$Bernoulli(m_i | \theta(x_i))$$

where m is the status of a tree being mature and θ , the probability of being mature, is some function of tree diameter, x . Recall that the Bernoulli is just a special case of the Binomial where the sample size is $n = 1$ (i.e. a single coin toss with some specified probability).

Now that we have specified the data models for our analysis and the process model for $g(x)$, the relationship between size and fecundity, we next need to specify the process model for $\theta(x)$, the relationship between tree size and maturation. For $\theta(x)$ we will need a function that spans the range of diameters and goes from 0 to 1 in the y axis.

One function that meets this criteria is the probit function, $probit(b_0, b_1)$ which has the same functional form as the normal CDF (`pnorm` in R). The probit has a nice intuitive interpretation in that the b_0 of the function, which is analogous to the mean, determines its midpoint (x value where the probability is 0.5) and the b_1 , which is analogous to the standard deviation, determines the steepness of the curve.

```
## example of probit
xseq = seq(0,30,length=200)
plot(xseq,pnorm(xseq,15,3),type='l')
```



Task 1

In fitting the model numerically we're going to need some initial conditions for the routine to start from. **Make a plot showing maturation (dia vs cones).** To this plot **add** $\theta(x)$ to the maturation plot. Choose the parameters for this function (b_0 , b_1) so that the plots is reasonable given the data. Specify the values for (b_0 , b_1) in R as a vector named "param".

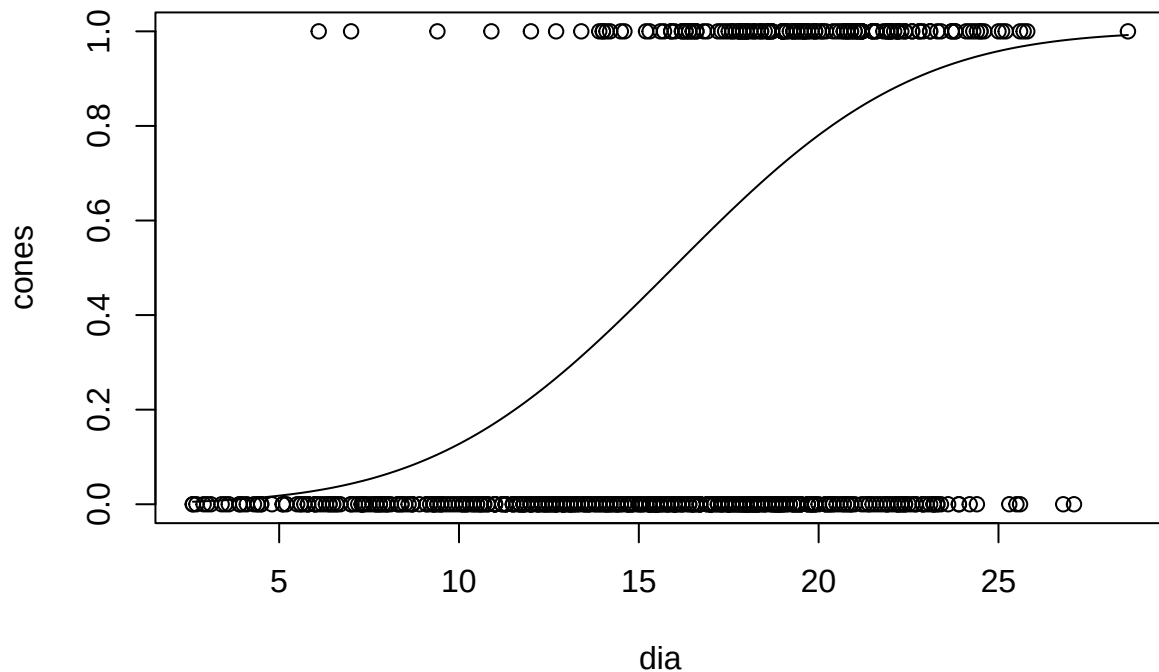
```
out.mat <- optimize(likMat,x=dia,p=cones)
```

```
out.mat
```

```
b0 = mean(dia)
b1 = sd(dia)
param = c(b0,b1)

xseq <- seq(min(dia), max(dia), length = 200)

plot(dia,cones)
lines(xseq, pnorm(xseq,b0,b1), type = 'l')
```



This vector will be passed to our likelihood function below. When choosing initial conditions, they don't have to be a great fit – finding that great fit is the job of the likelihood analysis – they just have to be in the right ballpark (e.g. your curves should show up on the plot of the data).

Task 2

- Similar to `likFec` and the models we constructed in the last tutorial, code up the negative log likelihood for this model as an R function

```
likMat <- function(param, x, m) {
  b0 <- param[1]
  b1 <- param[2]
  theta <- pnorm((x - b0) / b1)
  -sum(dbinom(m, size = 1, prob = theta, log = TRUE))
}
```

- Test your function at your initial guess to verify that the function is working

```
b0 = mean(dia)
b1 = sd(dia)
param = c(b0,b1)

likMat(param, x = dia, m = cones)
```

```
## [1] 442.0437
```

- Use a numerical optimization function to fit your log likelihood to the data. Make sure to use an optimization function that supports multivariate problems (not `optimize`)

```
out.mat <- optim(param, likMat, x = dia, m = cones)
out.mat
```

```
## $par
## [1] 21.794483  7.122348
##
## $value
## [1] 306.6578
##
## $counts
## function gradient
##      53      NA
##
## $convergence
## [1] 0
##
## $message
## NULL
```

Combined Fecundity and Maturation

If we observe cones on a tree ($y > 0$) it must be mature ($m = 1$) and thus the probability of this observation is

(1a)

$$p(y_i = 1, 2, \dots) = \text{Bernoulli}(m = 1|\theta(x_i)) \text{Pois}(y_i|g(x_i))$$

This likelihood involves both the probability that an individual is mature and the number of cones that were produced given that the individual was mature. The probability of zero cones involves two terms, that of an individual being immature and that of it being mature but with no cones are observed.

(1b)

$$p(y_i = 0) = \text{Bernoulli}(m = 0|\theta(x_i)) + \text{Bernoulli}(m = 1|\theta(x_i)) \text{Pois}(0|g(x_i))$$

The inclusion of these two terms in the model for zero counts makes it clear that both process models, $\theta(x)$ and $g(x)$, have to be fit simultaneously because fitting them individually (as we just did) misses the fact that some of the trees with a zero count are actually mature.

Lets put our two existing models together in order to define this more complex likelihood function. This function will include a likelihood that has two parts (when cone counts are zero vs positive) and a `return` statement that defines what the function returns when it is done. `return` statements are optional but useful for more complex functions in order to be explicit about what is being returned.

```
##' @param param  vector of model parameters
##' @param dia    tree diameter at breast height (cm)
##' @param ncone  cone counts (integer)
likfit = function(param,dia,ncone){
  a0 = param[1]
  b0 = param[2]
  b1 = param[3]
  cones = ncone > 0
```

```

## trees with cones
dia.cone <- dia[cones > 0]          ## find just the trees with cones
g.cone <- a0 * dia.cone^2          ## Fecundity fnc - g(x)
theta.cone <- pnorm(dia.cone,b0,b1,log.p=TRUE) ## maturation probit - theta(x)
prob.cone <- theta.cone + dpois(ncone[cones],g.cone,log=TRUE) ## lnL with cones

##trees with zero counts
dia.zero <- dia[cones == 0]        ## find the trees without cones
g.zero <- a0 * dia.zero^2          ## Fecundity fnc - g(x)
theta.zero <- pnorm(dia.zero,b0,b1) ## maturation probit - theta(x)
prob.zero <- log((1-theta.zero) + theta.zero*dpois(0,g.zero)) ## lnL without cones

return(-sum(prob.cone,prob.zero))  ## combined -lnL
}

```

Within this function the *dia.** lines define the subsets of data that are relevant for each condition. The *g.** line defines the expected value for $g(x)$, the number of cones per tree. The *theta.** line defines the θ function. The *prob.** line defines the log likelihood for each part corresponding to equations 1a and 1b above. Take a look at those functions and be sure you understand how they were converted into the calculations above. Finally, the `return` line calculates the total negative log likelihood by summing up the two terms of the likelihood function.

Having defined the likelihood function the next step is to numerically minimize the negative log likelihood

```

param <- c(0.008494147,21.794483,7.122348) ## initial guess (update this based on the first two mo
likfit(param,dia,ncone) ## verify that the function works

```

```
## [1] 1872.439
```

```

out = optim(param,          ## run numerical MLE
            likfit,
            lower=c(0.001,10,1),
            upper=c(1,30,10),
            method="L-BFGS-B",dia=dia,ncone=ncone)
out
            ## display results

```

```

## $par
## [1] 0.02351614 21.74063252 7.17872236
##
## $value
## [1] 1297.027
##
## $counts
## function gradient
##      25      25
##
## $convergence
## [1] 0
##
## $message
## [1] "CONVERGENCE: REL_REDUCTION_OF_F <= FACTR*EPSMCH"

```

```

a0 = out$par[1]          ## extract parameters for later use in plotting & analysis
b0 = out$par[2]
b1 = out$par[3]
lnL = out$value
n   = length(cones)

```

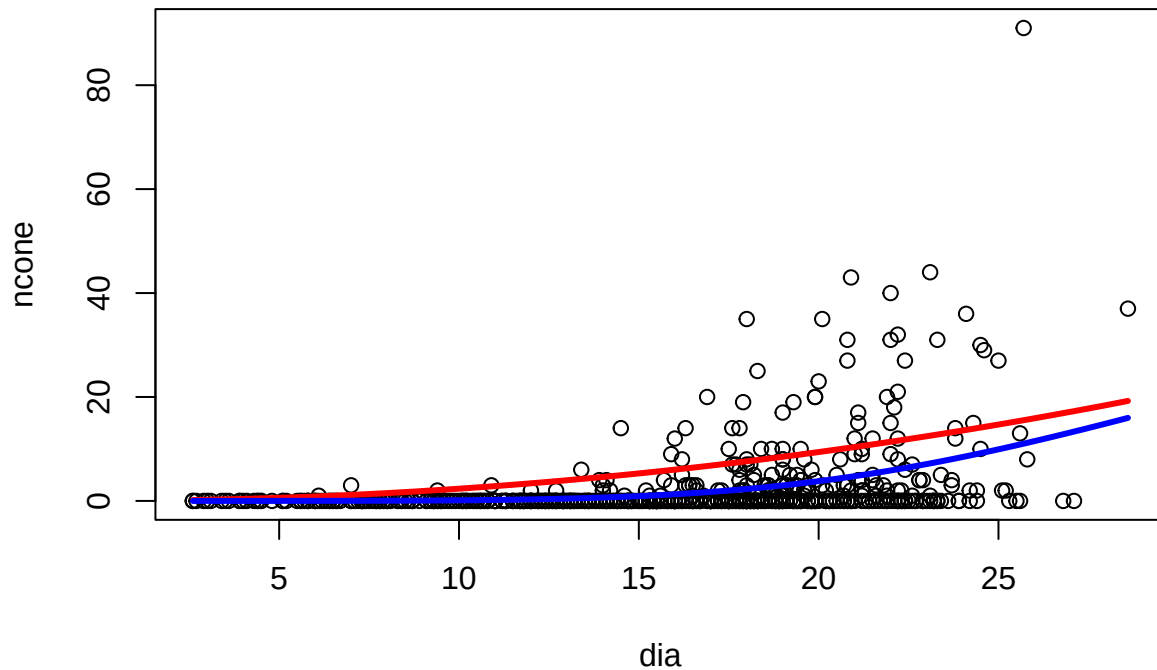
Task 3

Make two plots showing fecundity (dia vs ncone) and maturation (dia vs cones) showing both the raw data and the best fit model. For the fecundity plot include two lines, one representing fecundity vs size given maturity, $g(x)$, and the second that is the expected number of cones observed, which combines fecundity and the probability of maturation $g(x) \cdot \theta(x)$

```

plot(dia, ncone)
lines(xseq, out$par[1] * xseq^2, col = "red", lwd = 3)
lines(xseq, out$par[1] * xseq^2 * pnorm((xseq - out$par[2]) / out$par[3]),
      col = "blue", lwd = 3)

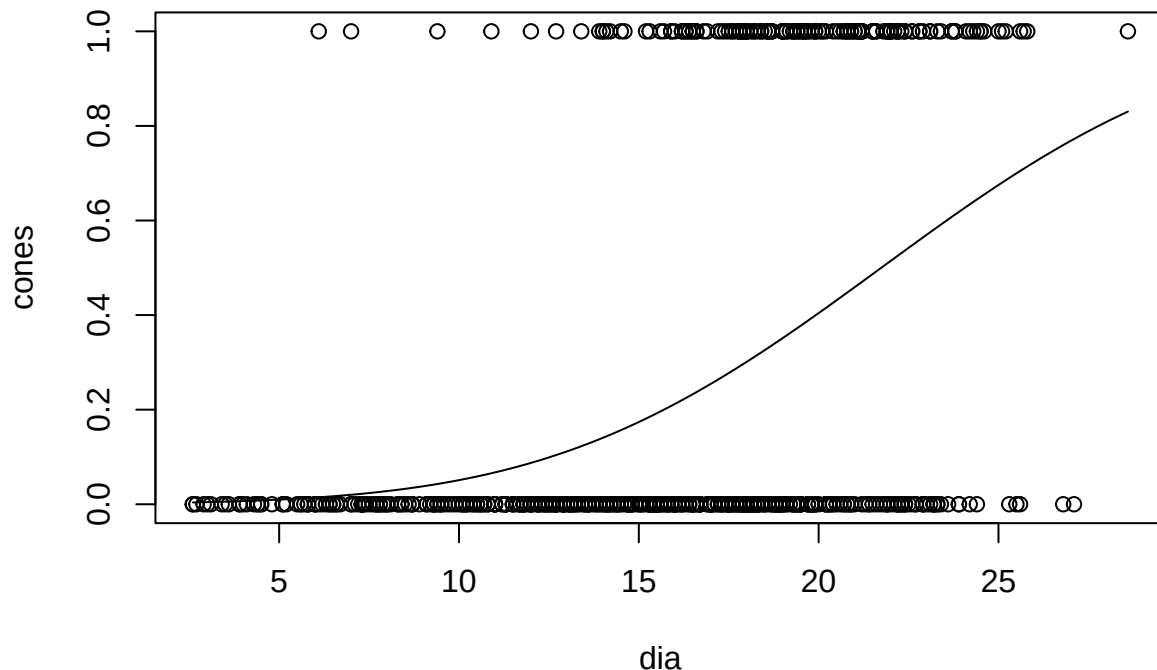
```



```

## Maturation
plot(dia, cones)
lines(xseq, pnorm(xseq, out$par[2], out$par[3]), type = 'l')

```



Effects of Elevated CO2 on Reproduction

Now, you may have noticed that the likelihood we wrote made no use of the CO2 treatment information. We began by fitting the model to the whole data set. We are next going to fit the model to the ambient and elevated data separately and then compare the fit of the data for the model with a treatment effect to the fit without the effect. In this approach we are choosing a biologically relevant null model – that fecundity and maturation vary with size but not CO2 – as opposed to simply testing whether each of the effects is different from zero. In the remainder of the tutorial we will review concepts related to hypothesis testing and model comparison.

To begin model comparison, let's look at how to fit the model to just the data from the ambient treatment. The first step of this is to extract the subset of data that we are interested in.

```
## Ambient
dia.amb = b0$diam[tmt=="AMB"] ## tree diameters
ncone.amb = b0$cones[tmt=="AMB"] ## number of cones
```

Next, let's fit the same likelihood model that we did previously but save the output to different variables. You'll notice that since we've now defined the data sets used by the `likfit` function differently all we have to do is re-fit the likelihood function as is.

```
out.amb = optim(param,likfit,method="L-BFGS-B",
               lower=c(0.001,10,1),upper=c(1,30,10),dia=dia.amb,ncone=ncone.amb)
a0[2] = out.amb$par[1]
b0[2] = out.amb$par[2]
```

```

b1[2] = out.amb$par[3]
lnL[2] = out.amb$value
n[2] = length(ncone.amb)
out.amb

```

```

## $par
## [1] 0.01882248 23.02788903 7.46780515
##
## $value
## [1] 522.0585
##
## $counts
## function gradient
##      41      41
##
## $convergence
## [1] 0
##
## $message
## [1] "CONVERGENCE: REL_REDUCTION_OF_F <= FACTR*EPSMCH"

```

You can compare the “out” and “out.amb” variables and see that the model has converged to different parameter values, which is to be expected since it is operating on a non-random subset of the full data. In storing the values from the optimization we’re converting each of the variables we defined earlier (a0, b0, etc.) into vectors and storing the values from this analysis as the second element in each of those vectors. This is done just to make comparing their values easier.

Task 4

Repeat the analysis we just did but this time fit the model for just the elevated CO2 treatment (tmt == CO2). Save the output to the variable “out.elev” and store the same output values to the third element of the parameter vectors (e.g. a0[3]). Generate a table of the values of the parameters and include it here.

```

## co2
dia.elev = b00$diam[tmt=="CO2"] ## tree diameters
ncone.elev = b00$cones[tmt=="CO2"] ## number of cones

out.elev = optim(param,likfit,method="L-BFGS-B",
                 lower=c(0.001,10,1),upper=c(1,30,10),dia=dia.elev,ncone=ncone.elev)
a0[3] = out.elev$par[1]
b0[3] = out.elev$par[2]
b1[3] = out.elev$par[3]
lnL[3] = out.elev$value
n[3] = length(ncone.elev)
out.elev

```

```

## $par
## [1] 0.02676403 20.64395429 6.77509672
##
## $value
## [1] 750.1744
##

```

```
## $counts
## function gradient
##      32      32
##
## $convergence
## [1] 0
##
## $message
## [1] "CONVERGENCE: REL_REDUCTION_OF_F <= FACTR*EPSMCH"
```

Model Selection

Finally, we'll perform a *Likelihood Ratio Test*, which is a statistical test that compares two nested models based on their deviance ($-2\ln L$). Nested models are models that have the same functional form but where the reduced version of the model is achieved by fixing parameter values a priori. For example, from the last tutorial the Weibull and Exponential are nested models, where setting the c parameter equal to 1 in the Weibull gives the Exponential. In the current context, the fit of the model to the whole dataset (out) is a nested version of the model with the CO2 treatment where the treatment effect is dropped. In this context we're considering the two fits of the data to the two treatments (out.amb and out.elev) as one single model that has the same sample size as the single fit but twice as many parameters. We haven't covered this, but deviance has a chi-square distribution with the degrees of freedom based on the difference in the number of parameters between two models. In this case the null model has 3 parameters (a_0 , b_0 , b_1) while the alternate model has six parameters because we have two a_0 's (ambient vs elevated), two b_0 's and two b_1 's.

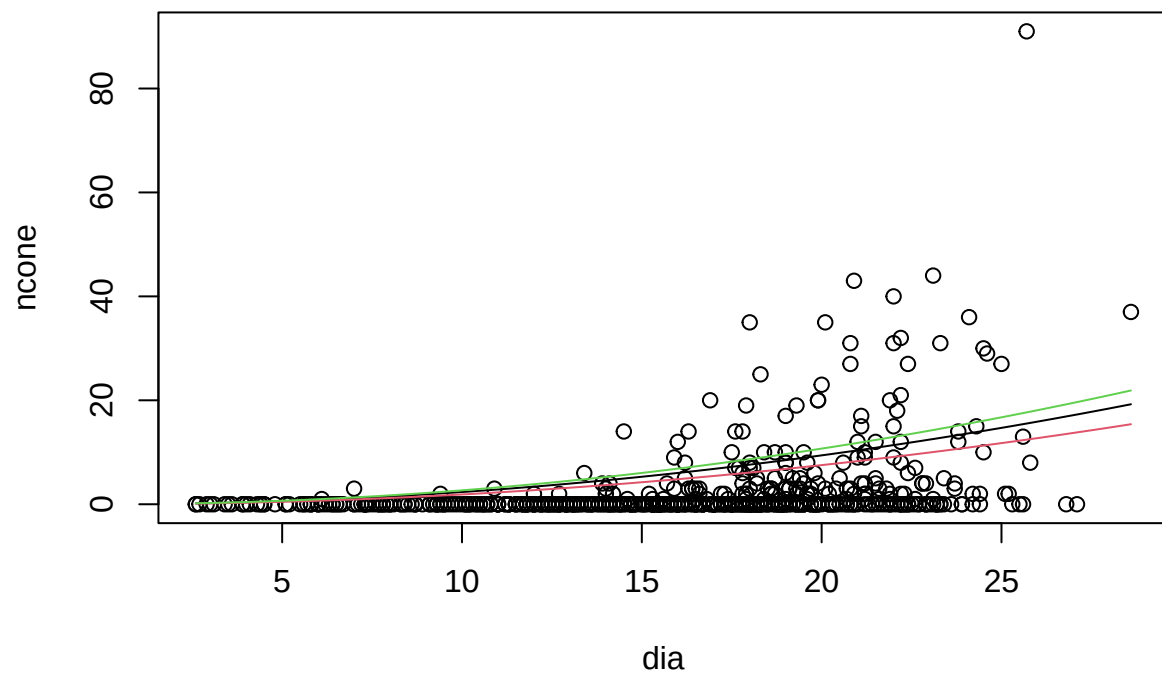
```
dev.null <- 2*lnL[1]          ## null model
dev.tmt  <- 2*lnL[2] + 2*lnL[3] ## model with elevated vs ambient CO2 treatment
LR       <- dev.null - dev.tmt  ## Likelihood ratio
pval     <- 1-pchisq(LR,3)
```

Since the p-value from this test is very small, for most any reasonable alpha (e.g. the traditional alpha = 0.05) we would take this as support for the alternate hypothesis that the treatment (CO2) has a significant effect on maturation and fecundity.

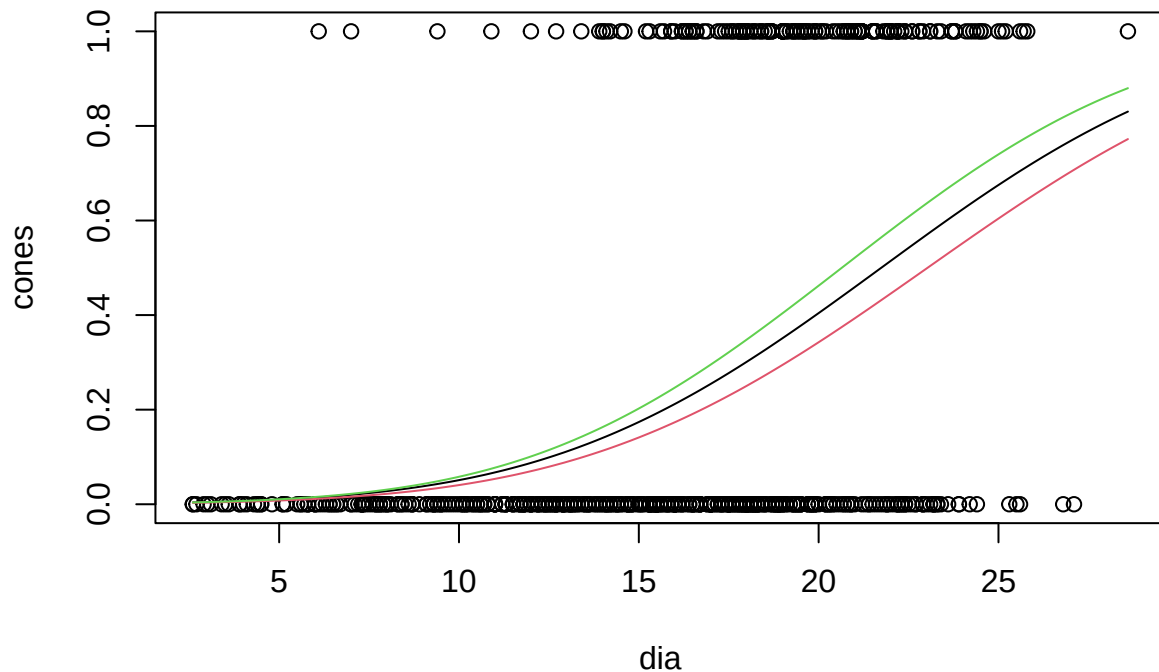
Task 5

Create figures analogous to Task 3 that show the fits for the full model, the ambient-only fit, and the elevated-only fit (i.e. 3 lines per graph, with different colors or symbols for ambient vs elevated data). Based on these graphs, the parameter values, and the likelihood ratio test, answer the following questions:

```
plot(dia, ncone)
lines(xseq, a0[1] * xseq^2,
      col = 1)
lines(xseq, a0[2] * xseq^2,
      col = 2)
lines(xseq, a0[3] * xseq^2,
      col = 3)
```



```
## Maturation
plot(dia,cones)
lines(xseq, pnorm(xseq,b0[1],b1[1]), type = 'l', col = 1)
lines(xseq, pnorm(xseq,b0[2],b1[2]), type = 'l', col = 2)
lines(xseq, pnorm(xseq,b0[3],b1[3]), type = 'l', col = 3)
```



1. Does elevated CO₂ have an effect on fecundity? (Provide explanation for your answer).

Yes, it increases expected cones at higher diameters. It also increases the likelihood of maturity given most diameters.

2. Is this effect simply due to there being larger trees in the elevated plots, or does CO₂ affect the maturation rate and/or the fecundity of trees (i.e. the cone production at a given tree size)? If there is an effect, in what direction is it?

There is a higher likelihood for a tree at any diameter to be mature (provide a cone). This model is saying that across all sizes, there is a higher likelihood that the tree has a cone. The same is true for number of cones. At all diameters, the elevated trees are expected to have more cones.

#Confidence and Predictive Intervals

Next, let's spend a bit of time applying the frequentist tools for generating confidence and predictive intervals and for selecting among models. For the model we just estimated, let us now estimate confidence intervals on the model parameters, as well as model confidence and predictive intervals, using bootstrap methods. We will also compare the results of the LRT with model selection using AIC.

Recall that we have now fit the pine cone count data to the whole data set (out), to just the ambient CO₂ data (out.amb) and to just the elevated CO₂ data (out.elev).

AIC vs Likelihood Ratio Test

Begin by calculating the AIC for the two models. We had previously stored the negative log likelihoods in a vector named lnL that contained the values for (combined, amb, elev). To estimate AIC we need to

know the total number of parameters for each model. For the model that analyzes the combined data there are 3 parameters, a_0 , b_0 , and b_1 . Because we did not use a Normal likelihood there's not an additional variance parameter that needs to be estimated. The model that treats ambient and elevated CO2 as separate treatments thus has 6 parameters. Recall that the formula for AIC is

$$AIC = -2\ln(L) + k \cdot n_{par}$$

Where: $\ln(L)$ is the log-likelihood of the fitted model. n_{par} is the number of parameters. $k = 2$ for the standard AIC.

Task 6 Compare the AICs for the two models, interpret the results for this test, and compare with the results from the LRT previously performed.

```
AIC_combined <- -2 * (-lnL[1]) + 2 * 3
AIC_separate <- -2 * (-lnL[2] + lnL[3])) + 2 * 6
AIC_combined
```

```
## [1] 2600.054
```

```
AIC_separate
```

```
## [1] 2556.466
```

The AIC of the separated models is significantly lower than the combined. This means that the treatment does meaningfully effect tree fecundity and maturation. This agrees with the LRT test that the treatment has a significant effect on maturation and fecundity.

Bootstrapped Interval Estimates

Next, let's calculate the bootstrap confidence and predictive intervals. This begins by setting up variables that control the sampling and storage for the output. Keep in mind that bootstrapping and MCMC are not the same thing. Each treats the data and the parameters differently – in the bootstrap we're generating random DATA while in MCMC the data are fixed but the parameters are random. This will make more sense when we cover MCMC.

For your report, make sure to set nboot to an appropriate number of replicates

```
## bootstrap
nboot <- 999          ## number of bootstrap samples
npred <- 31           ## number of X values you predict for
dseq <- seq(0,30,length=npred) ## diameter sequence
mle <- matrix(NA,nrow=nboot,ncol=3) ## storage for parameter estimates
conf.mat <- matrix(NA,nrow=nboot,ncol=npred) ## storage for maturation CI
conf.cone <- matrix(NA,nrow=nboot,ncol=npred) ## storage for fecundity CI
pred.mat <- matrix(NA,nrow=nboot,ncol=npred) ## storage for predictive maturation
pred.cone <- matrix(NA,nrow=nboot,ncol=npred) ## storage for predictive fecundity
```

Next we'll specify the data set that we'll be sampling from. In this case we'll begin by constructing interval estimates for the ambient CO2 data

```
## Ambient
dia.t <- b$diam[tmt=="AMB"] ## tree diameters
cones.t <- (b$cones > 0) ## whether cones are present
cones.t <- cones[tmt=="AMB"]
ncone.t <- b$cones[tmt=="AMB"] ## number of cones
```

Next is the main bootstrap loop. There are a number of steps to this process as we'll be generating both the parameters CI and the model CI and PI in one large loop. In the first part we define the resampled data set, making sure to sample so that we keep all xy pairs of data together. Next we'll fit the model to the resampled data using numerical optimization and store those parameters. After that we'll calculate the mean model prediction given the most recent set of parameters in order to calculate the model confidence interval. Finally we'll simulate pseudodata from the model to calculate the model predictive interval.

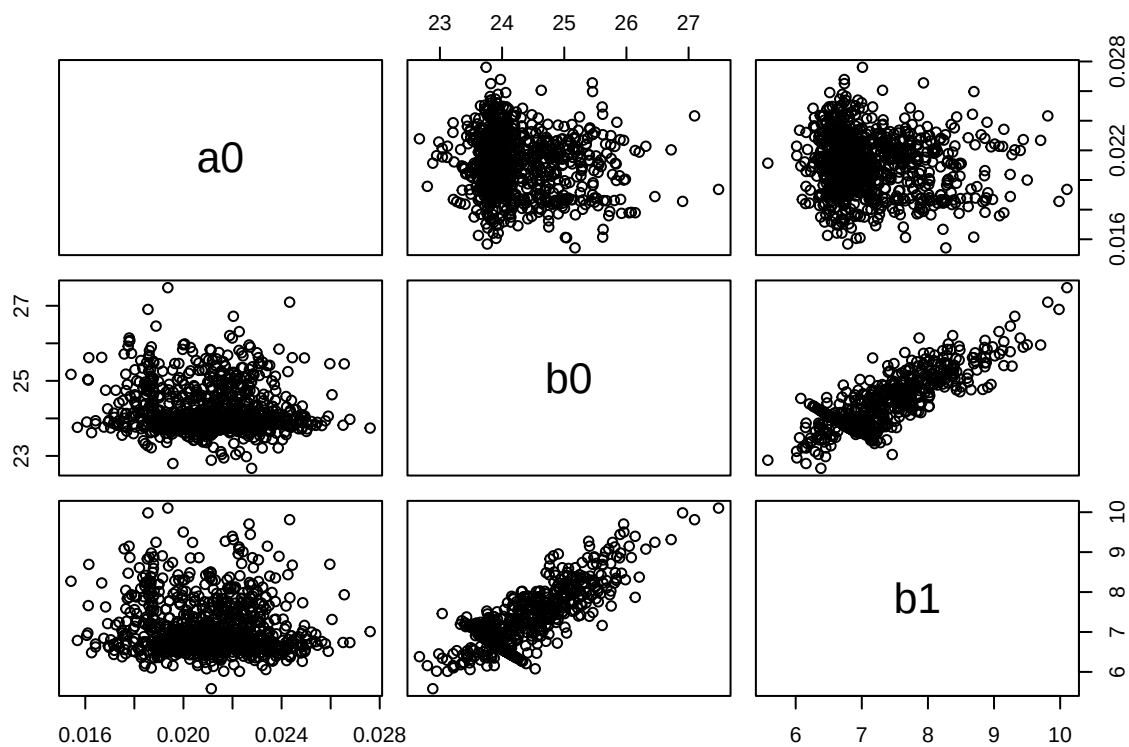
Don't forget to update the variable `param` so that it contains the previous best-fit parameters for the ambient CO2 model.

```
param <- c(0.01882248, 23.02788903, 7.46780515)
for(i in 1:nboot){
  if(i%100 == 0) print(i) ## progress indicator
  samp <- sample(length(dia.t),replace=T) ##Sample row indices
  out.boot <- optim(param,likfit,method="L-BFGS-B", ## fit model to sample
                    lower=c(0.001,10,1),upper=c(1,30,30),dia=dia.t[samp],ncone=ncone.t[samp])
  mle[i,] <- out.boot$par ## store parameters
}
```

```
## [1] 100
## [1] 200
## [1] 300
## [1] 400
## [1] 500
## [1] 600
## [1] 700
## [1] 800
## [1] 900
```

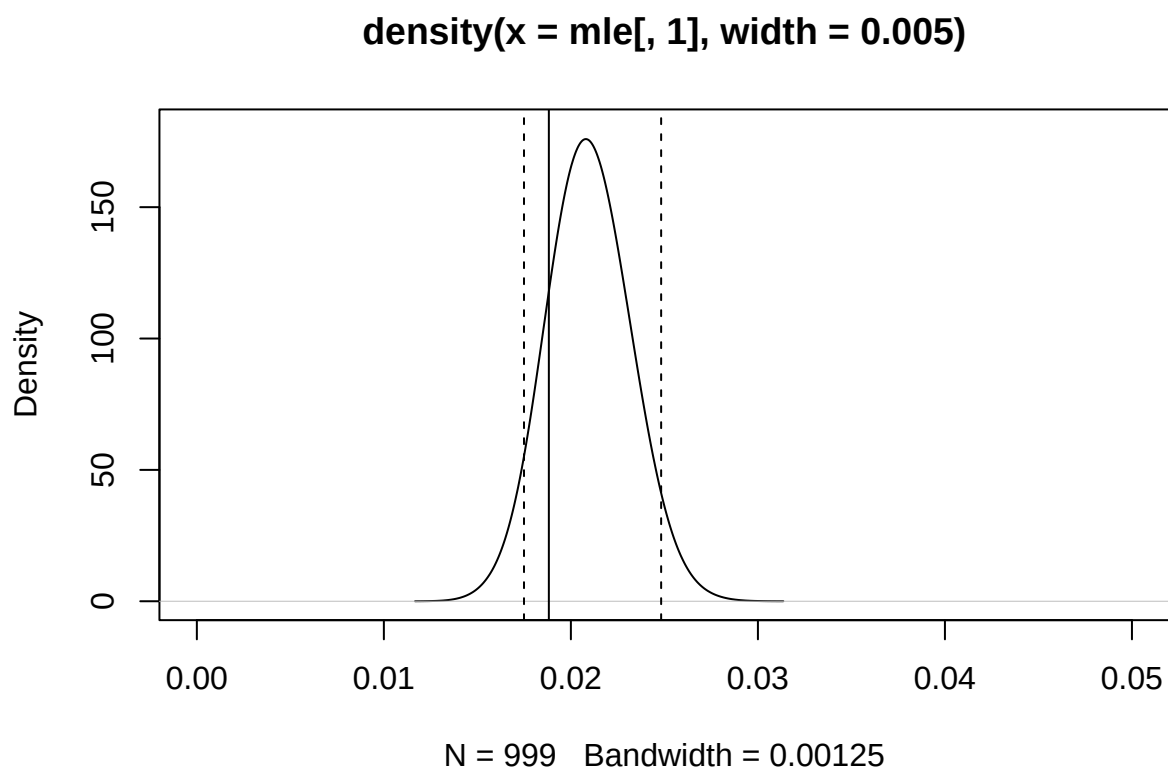
Once we've generated the bootstrap sample we'll want to begin by looking at the parameter level estimates. Lets first calculate the confidence intervals for each parameter using "quantile" and then use a pairs plot to assess parameter correlations.

```
colnames(mle) = c("a0","b0","b1")
a0.ci <- quantile(mle[,1],c(0.025,0.975))
b0.ci <- quantile(mle[,2],c(0.025,0.975))
b1.ci <- quantile(mle[,3],c(0.025,0.975))
pairs(mle)
```



Next, construct density plots for each of these parameters and add both the MLE and the confidence intervals.

```
## density plots
plot(density(mle[,1],width=0.005),type='l',xlim=c(0,0.05),ylim=c(0,180))
abline(v=a0.ci,lty=2)
abline(v=a0[2])
```

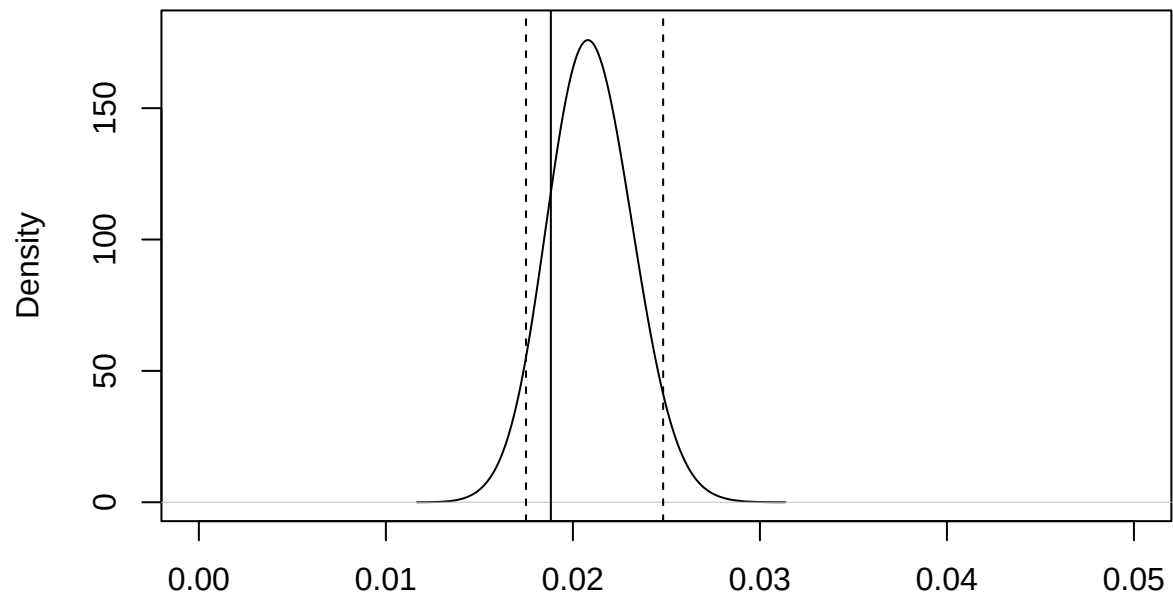


Task 7

- Generate density and CI plots for all 3 model parameters.

```
plot(density(mle[,1],width=0.005),type='l',xlim=c(0,0.05),ylim=c(0,180))
abline(v=a0.ci,lty=2)
abline(v=a0[2])
```

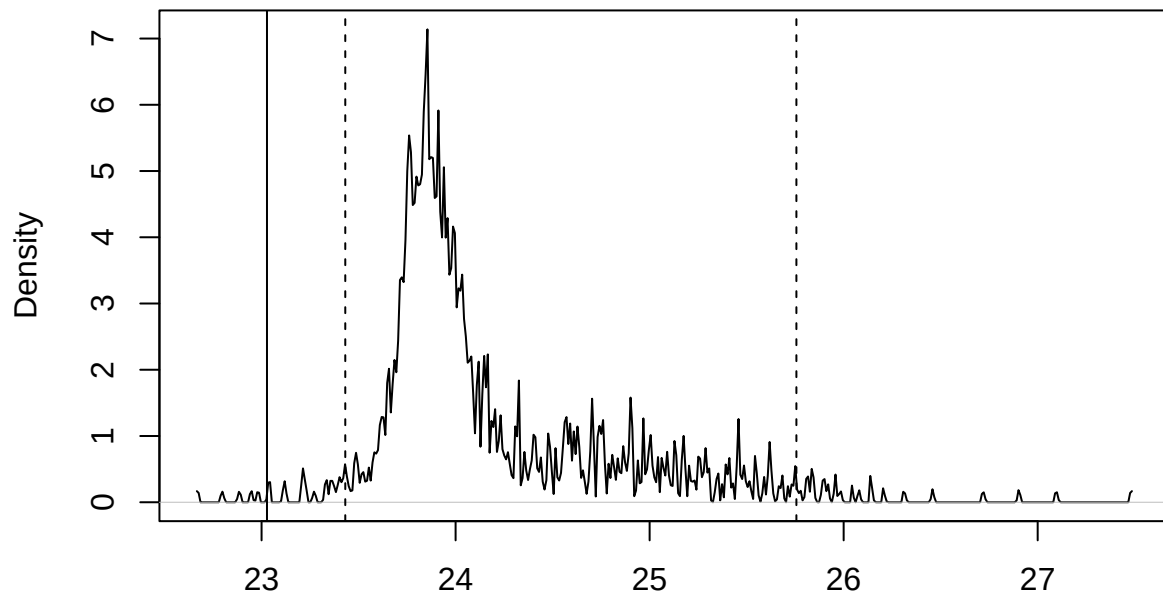
density(x = mle[, 1], width = 0.005)



N = 999 Bandwidth = 0.00125

```
plot(density(mle[,2],width=0.005),type='l')
abline(v=b0.ci,lty=2)
abline(v=b0[2])
```

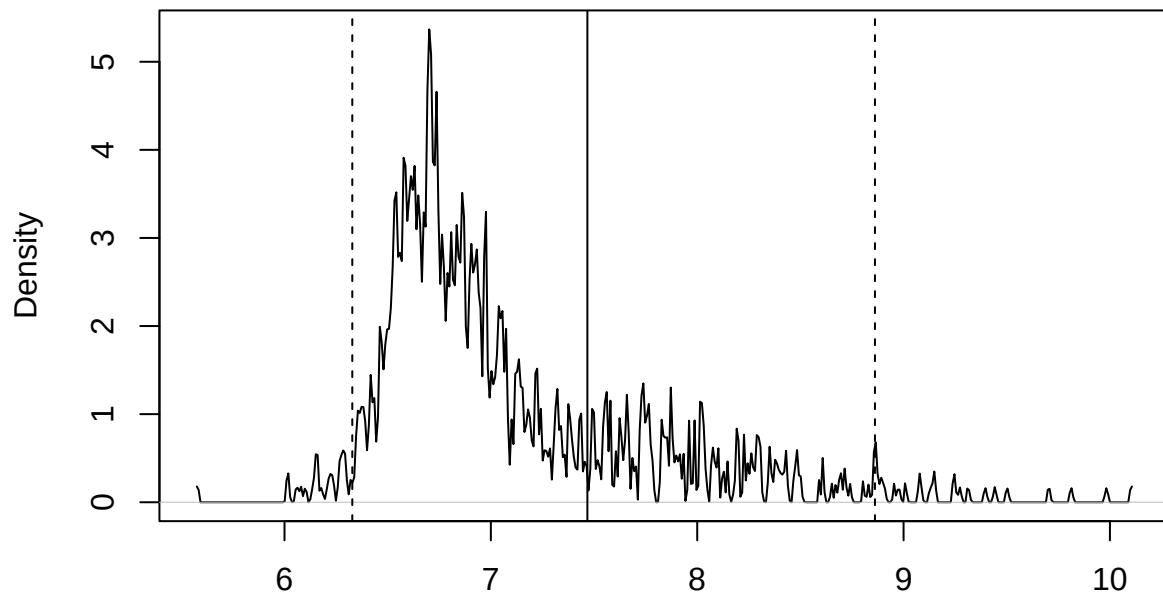
density(x = mle[, 2], width = 0.005)



N = 999 Bandwidth = 0.00125

```
plot(density(mle[,3],width=0.005),type='l')
abline(v=b1.ci,lty=2)
abline(v=b1[2])
```

density(x = mle[, 3], width = 0.005)



N = 999 Bandwidth = 0.00125

- Include a table of the three parameter estimates, standard errors, and CI.

```
estimates <- param

se <- apply(mle, 2, sd)

ci_lower <- c(a0.ci[1], b0.ci[1], b1.ci[1])
ci_upper <- c(a0.ci[2], b0.ci[2], b1.ci[2])

amb.table <- data.frame(
  Estimate = estimates,
  SE = se,
  CI_lower = ci_lower,
  CI_upper = ci_upper)

amb.table
```

```
##      Estimate      SE  CI_lower  CI_upper
## a0  0.01882248 0.001884637  0.01749227  0.02482588
## b0 23.02788903 0.613342289 23.43134694 25.75648907
## b1  7.46780515 0.650673584  6.32854509  8.86133864
```

- Use cov to generate an estimate of the parameter covariance matrix

```
cov.amb <- cov(mle)
cov.amb
```

```
##           a0           b0           b1
## a0  3.551856e-06 -5.959955e-05 -7.990691e-05
## b0 -5.959955e-05  3.761888e-01  3.262137e-01
## b1 -7.990691e-05  3.262137e-01  4.233761e-01
```

Task 8 Repeat this analysis but using the Fisher Information method. To do so, repeat the numerical optimization of the MLE *once* to the original data (not a full bootstrap) but including the argument `hessian = TRUE` when calling `optim`. Invert the Hessian to generate the covariance matrix. Construct confidence intervals for each parameter using the variances on the diagonal and report in a table. Compare this covariance matrix and associated parameter confidence interval estimates to Task 2.

```
fit.fisher <- optim(param,likfit,method="L-BFGS-B", ## fit model to sample
                    lower=c(0.001,10,1),upper=c(1,30,30),dia=diag(1),
                    ncone=ncone.t[samp], hessian = TRUE)

cov_fisher <- solve(fit.fisher$hessian)
cov_fisher
```

```
##           [,1]           [,2]           [,3]
## [1,]  3.371505e-07 -9.282254e-07 -2.274175e-06
## [2,] -9.282254e-07  2.982052e-01  2.441409e-01
## [3,] -2.274175e-06  2.441409e-01  3.286155e-01
```

```
se_fisher <- sqrt(diag(cov_fisher))

CI_lower_fisher <- estimates - 1.96 * se_fisher
CI_upper_fisher <- estimates + 1.96 * se_fisher

fisher.table <- data.frame(
  Parameter = c("a0", "b0", "b1"),
  Estimate = fit.fisher$par,
  SE = se_fisher,
  CI_lower = CI_lower_fisher,
  CI_upper = CI_upper_fisher)

fisher.table
```

```
##   Parameter   Estimate      SE   CI_lower   CI_upper
## 1      a0  0.02538976 0.0005806466  0.01768441  0.01996055
## 2      b0 23.81445783 0.5460816430 21.95756901 24.09820905
## 3      b1  6.75616292 0.5732499350  6.34423528  8.59137502
```

```
out.mat$par ## task 2
```

```
## [1] 21.794483  7.122348
```

```
fit.fisher$par
```

```
## [1] 0.02538976 23.81445783 6.75616292
```

Task 2 only had two parameters. Comparing them to the fisher method, b0 is more different than b1. this difference could be due to the fact that now we are taking into account cone counts as well.

Model Intervals

Let's now move on to constructing the model confidence intervals and predictive intervals. Since we have two process models for this analysis we'll start with the intervals on the maturation model. First, we'll want to generate the predictions

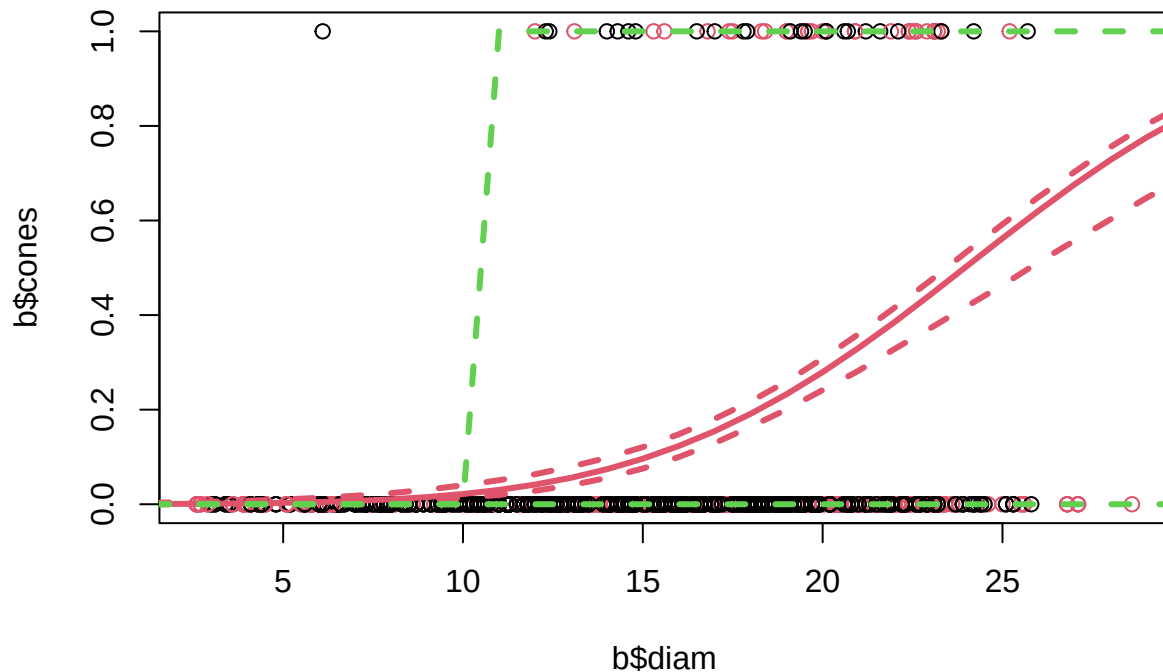
```
for(i in 1:nboot){  
  conf.mat[i,] <- pnorm(dseq,mle[i,2],mle[i,3])      ## store model / parms  
  pred.mat[i,] <- rbinom(npred,1,conf.mat[i,])  
}
```

Next we'll want to generate the CI and PI from the full distribution of predictions

```
ci.mat <- apply(conf.mat,2,quantile,c(0.025,0.5,0.975))  
pi.mat <- apply(pred.mat,2,quantile,c(0.025,0.975))
```

Finally, lets plot the model confidence and predictive intervals and compare them to the data

```
## maturation plot  
cones = b$cones > 0  
plot(b$diam,b$cones,col = as.factor(b$tmt),ylim=c(0,1))  
lines(dseq,ci.mat[2,],col=2,lwd=3)      ## median model  
lines(dseq,ci.mat[1,],col=2,lty=2,lwd=3)  ## 95% CI  
lines(dseq,ci.mat[3,],col=2,lty=2,lwd=3)  
lines(dseq,pi.mat[1,],col=3,lty=2,lwd=3)  ## 95% PI  
lines(dseq,pi.mat[2,],col=3,lty=2,lwd=3)
```



Task 9

Generate and plot the confidence and predictive interval for the full fecundity process model (which combines both maturation and fecundity|maturation). Include the observed data in the plot and interpret both this plot and the maturation plot in your answer.

```
for(i in 1:nboot){
  a0_i <- mle[i,1]
  b0_i <- mle[i,2]
  b1_i <- mle[i,3]

  g_i      <- a0_i * dseq^2
  theta_i  <- pnorm((dseq - b0_i) / b1_i)
  mu_i     <- g_i * theta_i

  conf.cone[i,] <- mu_i
  pred.cone[i,] <- rpois(npred, lambda = mu_i)
}

ci.cone <- apply(conf.cone, 2, quantile, c(0.025, 0.5, 0.975))
pi.cone <- apply(pred.cone, 2, quantile, c(0.025, 0.975))

plot(dia.t, nccone.t, col = as.factor(b$tmt))

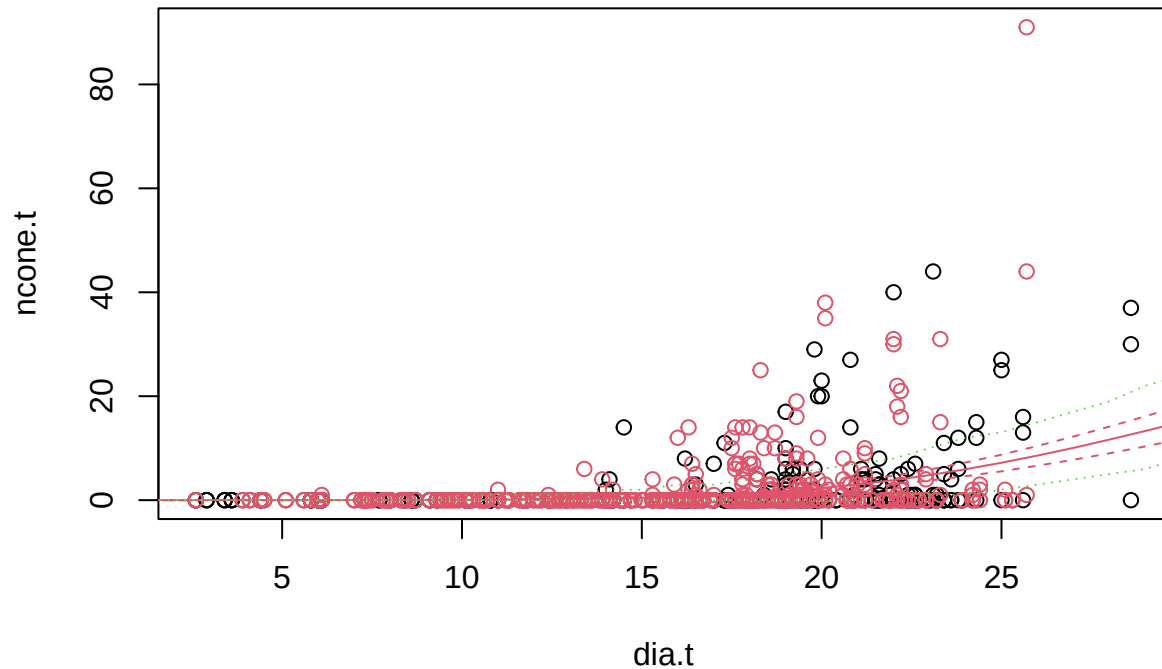
lines(dseq, ci.cone[2,], col=2)

lines(dseq, ci.cone[1,], col=2, lty=2)
```

```
lines(dseq, ci.cone[3,], col=2, lty=2)
```

```
lines(dseq, pi.cone[1,], col=3, lty=3)
```

```
lines(dseq, pi.cone[2,], col=3, lty=3)
```



The predictive interval is wider than the confidence interval, especially at higher diameters. This makes sense especially looking at the earlier maturation graph. There the predictive interval is huge, and this plot uses maturation. I see a good spread in treatment vs not.

Task 10

Repeat the bootstrap analysis for the elevated CO2 treatment. Include the figures and tables from tasks 2 & 3 for the elevated plots as well as a final plot that compares the model confidence intervals between the elevated and ambient data on one figure

```
nboot <- 999
npred <- 31
dseq <- seq(0, 30, length = npred)

mle.e <- matrix(NA, nrow = nboot, ncol = 3)
conf.mat.e <- matrix(NA, nrow = nboot, ncol = npred)
conf.cone.e <- matrix(NA, nrow = nboot, ncol = npred)
pred.mat.e <- matrix(NA, nrow = nboot, ncol = npred)
pred.cone.e <- matrix(NA, nrow = nboot, ncol = npred)

dia.e <- b$diam[tmt == "CO2"]
```

```

cones.e <- (b$cones > 0)[tmt == "CO2"]
ncone.e <- b$cones[tmt == "CO2"]

param.e <- c(0.02676403, 20.64395429, 6.77509672) ##update to elev

for(i in 1:nboot){
  samp <- sample(length(dia.e), replace = TRUE)
  out.boot <- optim(param.e, likfit, method = "L-BFGS-B",
                    lower = c(0.001, 10, 1), upper = c(1, 30, 30),
                    dia = dia.e[samp], ncone = ncone.e[samp])
  mle.e[i,] <- out.boot$par
}

a0.ci.e <- quantile(mle.e[,1], c(0.025, 0.975))
b0.ci.e <- quantile(mle.e[,2], c(0.025, 0.975))
b1.ci.e <- quantile(mle.e[,3], c(0.025, 0.975))

for(i in 1:nboot){
  conf.mat.e[i,] <- pnorm(dseq, mle.e[i,2], mle.e[i,3])
  pred.mat.e[i,] <- rbinom(npred, 1, conf.mat.e[i,])
}

ci.mat.e <- apply(conf.mat.e, 2, quantile, c(0.025, 0.5, 0.975))
pi.mat.e <- apply(pred.mat.e, 2, quantile, c(0.025, 0.975))

for(i in 1:nboot){
  a0_i <- mle.e[i,1]
  b0_i <- mle.e[i,2]
  b1_i <- mle.e[i,3]

  g_i <- a0_i * dseq^2
  theta_i <- pnorm((dseq - b0_i) / b1_i)
  mu_i <- g_i * theta_i

  conf.cone.e[i,] <- mu_i
  pred.cone.e[i,] <- rpois(npred, lambda = mu_i)
}

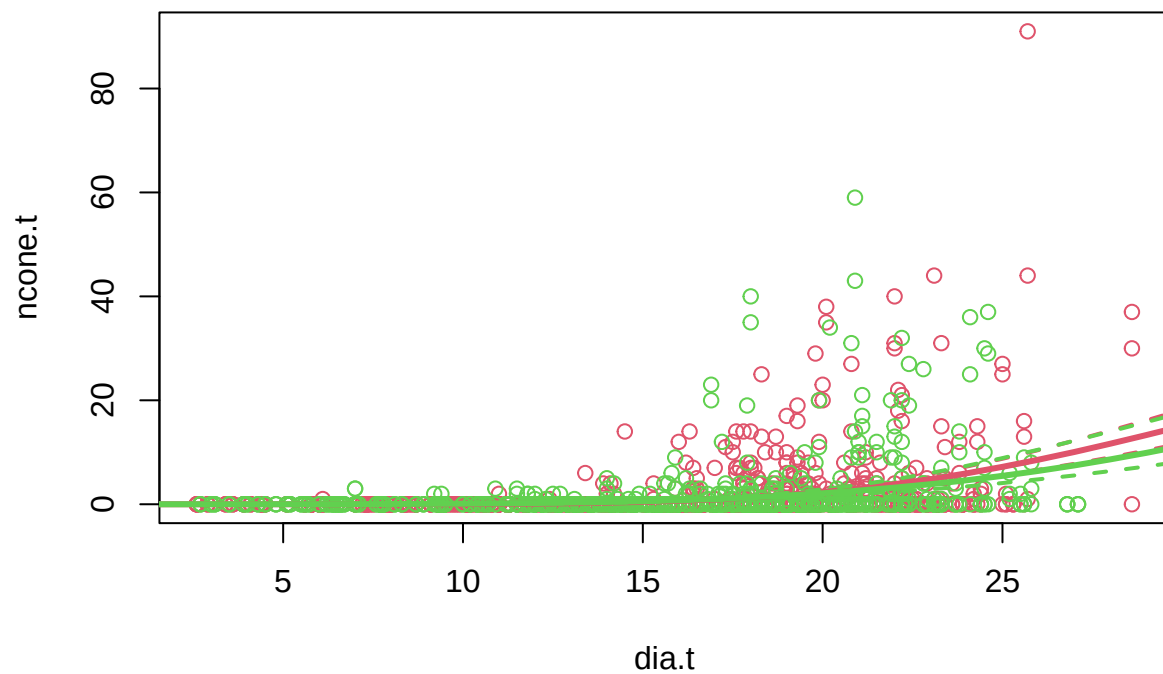
ci.cone.e <- apply(conf.cone.e, 2, quantile, c(0.025, 0.5, 0.975))
pi.cone.e <- apply(pred.cone.e, 2, quantile, c(0.025, 0.975))

plot(dia.t, ncone.t, col = 2)
points(dia.e, ncone.e, col = 3)

lines(dseq, ci.cone[2,], col = 2, lwd = 3)
lines(dseq, ci.cone[1,], col = 2, lty = 2, lwd = 2)
lines(dseq, ci.cone[3,], col = 2, lty = 2, lwd = 2)

lines(dseq, ci.cone.e[2,], col = 3, lwd = 3)
lines(dseq, ci.cone.e[1,], col = 3, lty = 2, lwd = 2)
lines(dseq, ci.cone.e[3,], col = 3, lty = 2, lwd = 2)

```



AI Statement: I found questions 8-10 to be extremely difficult. AI helped me to get to the answers, but I could not have gotten there myself. I tried to make sure I understood each step and could write out each line and understand before I moved on.